
boolean



QUERIES

Utilizziamo il Prisma Client per interagire con il database

Prisma JS

Installiamo l'ultima dipendenza di progetto Prisa

```
1 pnpm add @prisma/client-runtime-utils@7.10.1
```

Questo comando permette una corretta gestione per l'utilizzo del client prisma con il comando `pnpm watch` all'interno del nostro progetto Express.

Prisma JS

Come utilizzare il Prisma Client

Per specificare su quale modello desideriamo effettuare le operazioni, utilizziamo il **nome del modello** (in formato camelCase) come proprietà dell'oggetto istanziato dalla classe PrismaClient.

Nel nostro caso, utilizzeremo **prisma.product**, seguito dal nome del metodo che corrisponde al tipo di operazione che desideriamo eseguire.

Tutte le operazioni che eseguiamo sul database tramite il Prisma Client sono asincrone.

CRUD

Index: leggere più record dalla tabella

Per leggere tutti i prodotti dalla tabella abbiamo a disposizione il metodo **findMany**:

```
1 import prisma from '../lib/prisma.js';
2
3 // INDEX
4 router.get('/', async (req, res) => {
5   const products = await prisma.product.findMany();
6
7   res.json(products);
8 });
```

CRUD

Show: leggere un record specifico dalla tabella

Per leggere un prodotto specifico dalla tabella abbiamo a disposizione il metodo **findUnique** passando un oggetto con la proprietà **where** e il valore del campo che vogliamo recuperare.

```
1  import prisma from '../lib/prisma.js';
2
3  // SHOW
4  router.get('/:id', async (req, res) => {
5    const id = Number(req.params.id);
6    console.log(id, typeof id);
7
8    const product = await prisma.product.findUnique({
9      where: {
10       id,
11     },
12   });
13
14   res.json(product);
15 });
```

Relation queries

Inclusione dei record correlati durante la lettura

Per incorporare i dati di una tabella in relazione con la tabella attuale nella query Prisma mette a disposizione le seguenti proprietà:

- **include** per incorporare i record correlati.
- **select** inserito all'interno di include per selezionare solo determinati campi dei record correlati.

```
1  import prisma from '../lib/prisma.js';
2
3  // SHOW
4  router.get('/:id', async (req, res) => {
5    const id = Number(req.params.id);
6
7    const product = await prisma.product.findUnique({
8      where: {
9        id,
10     },
11     include: {
12       // Tutte le colonne
13       category: true,
14
15       // Oppure solo colonne specifiche
16       category: {
17         select: {
18           id: true,
19           name: true,
20         },
21       },
22     },
23   });
24
25   res.json(product);
26 });
```

CRUD

Store: creazione di un nuovo record

Per creare un nuovo record in una tabella, Prisma ci mette a disposizione il metodo **create**.

Al metodo dovremo passare un oggetto che contiene la proprietà **data**, che a sua volta conterrà tutti i dati necessari per creare la nuova pizza.

Se la creazione avviene con successo, verrà restituito l'oggetto appena creato con il suo ID.

```
1  import prisma from '../lib/prisma.js';
2
3  // STORE
4  router.post('/', async (req, res) => {
5    const { name, description, price,
6            quantity, categoryId } = req.body;
7
8    const newProduct = await prisma.product.create({
9      data: {
10       name,
11       description,
12       price,
13       quantity,
14       categoryId,
15     },
16     select: {
17       id: true,
18     },
19   });
20
21   res.status(201).json(newProduct);
22 });
```

CRUD

Delete: cancellare un record

Per cancellare una record presente nella tabella, utilizziamo il metodo **delete**, passando un oggetto che include la proprietà **where** con il campo id del prodotto che vogliamo cancellare.

```
1  // DELETE
2  router.delete('/:id', async (req, res) => {
3    const id = Number(req.params.id);
4
5    await prisma.product.delete({
6      where: {
7        id: id,
8      },
9    });
10
11   res.sendStatus(204);
12 });
```

CRUD

Update: aggiornare un record

Per modificare un record già presente nella tabella, possiamo utilizzare il metodo **update.**

Dovremo passare un oggetto che include:

- La proprietà **where** con il valore del record da modificare
- La proprietà **data** con i nuovi dati da applicare alla pizza

```
// UPDATE
router.patch('/:id', async (req, res) => {
  const id = Number(req.params.id);

  const { name, description, price, quantity,
          isActive, categoryId } = req.body;

  const updatedProduct = await prisma.product.update({
    where: {
      id: id,
    },
    data: {
      name,
      description,
      price,
      quantity,
      isActive,
      categoryId,
    },
  });

  res.json(updatedProduct);
});
```

ESERCITAZIONE